מבוא למדעי המחשב - 67101 - סמסטר ב' 21/22
מרצה: ד"ר אריה שלזינגר
מועד ב' - 29.7.22

<u>חלק א' - הצעות לפתרון:</u>

א'.1

```
Space Complexity: O(n*m)
Time Complexity:  O(n*m*log(n*m))
```

ב'.1

```
O(n^3)
```

א'.2

```
def go_back(lg):
    if lg:
        yield from go_back(lg.next)
        yield lg.data
```

ב'.2

```
def swap_next(lng):
    while lng is not Lnk.empty and lng.next is not Lnk.empty:
        lng.data, lng.next.data = lng.next.data, lng.data
        lng = lng.next.next
```

א'.3

```
def add_pos_evens(liz):
    if len(liz) == 0:
        return 0
    else:
        if liz[0] % 2 == 0 and liz[0] >= 0:
            return liz[0] + add_pos_evens(liz[1:])
        else:
            return add_pos_evens(liz[1:])
```

ב'.3

```
def high_low(n):
    if n < 10:
        return True
    elif n % 10 > (n // 10) % 10:
        return False
    else:
        return high_low(n // 10)
```

4.א'

```
def add_sparse_matrices(A, B, dim):
    ans = []
    for i in range(dim[0]):
        ans.append([0] * dim[1])

    for coords, value in A.items():
        ans[coords[0]] [coords[1]] += value

    for coords, value in B.items():
        ans[coords[0]] [coords[1]] += value

    return ans
```

4.ב'

```
def two_diags(mat):
    if len(mat) == 1:
        return mat[0][0]

    diag1,diag2 = 0,0
    for r in range(len(mat)):
        diag1 += mat[r][r]
        diag2 += mat[r][-r-1]

    return diag1, diag2
```

חלק ב' - הצעות לפתרון:

.5

```
def place_between(inter, lnk):
    if lnk is Lnk.empty:
        return Lnk.empty
    elif lnk.next is Lnk.empty:
        return Lnk(lnk.data)
    elif lnk.data == lnk.next.data:
        return Lnk(lnk.data, Lnk(inter, place_between(inter, lnk.next)))

    return Lnk(lnk.data, place_between(inter, lnk.next))
```

.6

```
def no_twins(n,x,y):

    if n == 0:
        return [[]]
    elif n == 1:
        return [[x], [y]]
    else:
        k1, k2 = no_twins(n-1,x,y), no_twins(n-2,x,y)

    return [[x] + d for d in k1] + [[y, x] + d for d in k2]
```

.7

```
def ops_on(oper , vals):
    if oper == '+':
        return sum(vals)
    elif oper == '*':
        return mult(vals)

def mult(vals):
    res = 1
    for x in vals:
        res *= x
    return res

def ops_to_leaves(iTree):

    if iTree.data in ('*', '+'):
        operands = []
        for br in iTree.branches:
            operands.append(ops_to_leaves(br))
        iTree.branches = [Tree(iTree.data)] + iTree.branches
        iTree.data = ops_on(iTree.data, operands)
    return iTree.data
```

.8

```
def pardes (lst, n=0):
    if n >= len(lst) or lst[n] is None:
        return BinT.empty

    left, right = pardes(lst, 2*n+1), pardes(lst, 2*n+2)
    return BinT(lst[n], left, right)
```

.9

```
def how_close(tr):
    near = abs(tr.data - sum([subtr.data for subtr in tr.branches]))
    return min([near] + [how_close(subtr) for subtr in tr.branches])
```

.10

```
def single_rep(lg):
    if lg is Lnk.empty:
        return 0
    elif lg.next is Lnk.empty:
        lg.next = Lnk(lg.data)
        return 1
    elif lg.data == lg.next.data:
        return single_rep(lg.next.next)
    else:
        lg.next= Lnk(lg.data , lg.next)
        return 1 + single_rep(lg.next.next)
```